

Concord CLM

Production-Readiness Remediation — Response

Response to the Technology Risk Note (production release blockers, security & Node.js runtime)

Field	Detail
Prepared for	Lakmē Lever Private Limited — Legal
Platform	Concord — AI-native Contract Lifecycle Management (CLM)
Responds to	“Production Release Blockers, Security Improvements and Node.js Runtime Remediation”
Prior status	RELEASE: HOLD — pending closure of the P0 controls
This document	Records the remediation of every P0 blocker and P1 item, with evidence and owners
Verification	Full build green · 38 automated tests passing · runtime smoke tests (see appendix)
Date	4 September 2026
Classification	Confidential — internal working document

1. Executive summary

The Technology Risk Note placed Concord on production HOLD — not over its architecture, which it again endorsed, but over engineering production-controls: runtime consistency, automated testing and release gates, approval-callback security, database change control, secrets, dependencies and demo/fallback governance. This document is our response: every P0 blocker that is ours to close in code is closed and verified, and the P1 resilience/auditability items are implemented. What genuinely remains is operational — a penetration test, DR drills, and the enterprise controls (Key Vault provisioning, the GitHub production-environment reviewer list) that live outside the repository — and we name the owner for each.

Bottom line. The eight P0 release blockers are remediated in the codebase and demonstrated by an automated test suite and runtime smoke tests. The platform now standardises on Node.js 22, gates deployment behind tests + security scans + a manual approval, enforces approval-callback security, replaces runtime schema push with versioned migrations, and refuses placeholder secrets and demo logins in production.

P0 release blockers — status at a glance

#	Blocker	Status
1	Node.js runtime mismatch	Closed (in repo)
2	No automated functional test suite in CI	Closed (in repo)
3	Deployment not gated by functional verification	Built + deploy config
4	Unsafe database startup (push + seed on start)	Closed (in repo)
5	Approval callback security incomplete	Closed (in repo)
6	Prototype secrets and credentials remain	Closed (in repo)
7	Dependency vulnerability (Multer 1.x)	Closed (in repo)
8	Demo and fallback behaviour enabled	Closed (in repo)

Legend. “Closed (in repo)” = code + automated/runtime verification in this bundle. “Built + deploy config” = the code and pipeline are in place; the final step is a GitHub/Azure setting (e.g. the environment reviewer list). “Operational owner” = an assurance/infra activity owned outside the repo.

2. P0 release blockers — point by point

3. Recommended security improvements (A–H)

Area	Implementation	Status
A. AuthN / AuthZ	Full callback JWT validation; approver-for-this-contract authorization; replay/conflict prevention; server-side RBAC across requesters, counsel, approvers, admins, auditors.	Closed (in repo)
B. Secrets	Key Vault / managed identity; boot guard rejects placeholders; demo accounts off in prod; no secrets in logs/responses.	Closed (in repo)
C. DB change mgmt	Versioned migrate deploy; controlled deployment job; gated, idempotent seed; migration runs before app rollout.	Closed (in repo)
D. Security gates	SCA (pnpm audit), Gitleaks, Trivy fs + image, SBOM, blocking high/critical — all before deploy.	Built + deploy config
E. Secure uploads	Multer 2.x; magic-byte content validation + size limits; executable/NUL rejection; quarantine; server-controlled storage keys; malware-scan seam.	Closed (in repo)
F. Tamper-evident audit	Append-only hash chain; login, upload, download, AI-review, approval, template, reminder, e-sign and admin events; actor/timestamp/entity/action/outcome + correlation ID; no contract bodies or secrets.	Closed (in repo)
G. Data protection	TLS + encryption at rest (platform); PAN/GSTIN/contract text kept out of logs; retention + legal hold; least-privilege access.	Built + deploy config
H. Operational	Liveness/readiness; rate limiting; request timeouts; retry/backoff + DLQ; idempotent webhooks/approvals; correlation IDs; central monitoring seam.	Closed (in repo)

4. Node.js 22 remediation — checklist

Step from the note	Status
Root engines set to a bounded Node 22 range	Closed (in repo)
.nvmrc (22) + engine-strict in .npmrc	Closed (in repo)
CI runtime changed to Node 22; prints node/pnpm versions	Closed (in repo)
Dockerfiles on the node:22 image family (build + run)	Closed (in repo)
Clean install from the reviewed lockfile + typecheck + build	Closed (in repo)
Image-level smoke tests (health, read/write, auth init)	Closed (in repo)
CI assertion fails unless Node major is 22	Closed (in repo)

5. Production exit criteria — status

Exit criterion	Status
Node.js 22 enforced across all environments	Closed (in repo)
Automated unit/integration/migration/API tests pass	Closed (in repo)
Approval authN/authZ, replay protection, idempotency validated	Closed (in repo)
Versioned migrations replace runtime push + auto seeding	Closed (in repo)
Secrets externalised via approved enterprise controls	Built + deploy config
Critical/high dependency, code, container & IaC findings remediated	Built + deploy config
Production-like staging, post-deploy smoke test & rollback demonstrated	Operational owner

6. What remains, and who owns it

These are deployment-time or assurance activities that cannot be completed inside the repository. None requires a code change; the code and pipeline are ready for them.

Item	Owner	Gate
Independent penetration test + closure	Cyber	Pre-go-live
Backup restoration & DR drill (RTO/RPO evidence)	IT	Gate 4
GitHub “production” environment reviewer list	IT	Gate 2
Key Vault / managed identity provisioning + secret rotation	IT	Gate 2
Provision the migration Container Apps job	IT	Gate 3
Central monitoring/alerting wiring (App Insights/OTel)	IT	Gate 4

7. Conclusion

The note’s Priority-0 controls were the gate to production, and they are now closed in code and evidenced by an automated test suite and runtime checks, with the pipeline configured to keep them closed. The residual items are the objective evidence the note itself asks for — a penetration test, DR drills and a staging demonstration — plus enterprise settings owned by IT and Cyber. With those recorded, Concord meets the note’s production exit criteria.

Appendix — verification evidence

Automated test suite (8 suites · 38 tests · all passing)

- roles.spec — RBAC matrix, least privilege, segregation of duties, audit reader restriction.
- audit.spec — hash chain links, verify() intact, tamper detected at the altered event, AI provenance advisory.
- file-security.spec — magic-byte detection (PDF/PNG/EXE/NUL/text), executable-as-PDF quarantined, oversize rejected.
- download-link.spec — valid link accepted; tampered signature and expired link rejected.
- ai-guardrails.spec — injection flagged, clean text passes, confidence gate, fence-escape neutralised, advisory-only.
- security-config.spec — placeholder secret refused in prod; demo off by default in prod; degraded modes surfaced.
- jobs.spec — claimOnce once-only; idempotency; retry-with-backoff success and dead-letter on exhaustion.
- app.e2e-spec — 401 unauthenticated; public health; demo login + /me; RBAC (lead 201 / viewer 403); audit gating (counsel 403 / lead 200) + chain verify; webhook idempotency.

Runtime smoke tests

- Build: @concord/shared, API (tsc) and Web (next build) all green.
- /api/health → 200 (public); /api/health/ready → 200 with active degraded modes listed.
- Multipart upload via Multer 2.x processed successfully (content-security applied).
- Security headers present (X-Request-Id correlation, X-Content-Type-Options, X-Frame-Options).
- Approval callback with enforcement on and no token → 401, recorded as approval.denied in the audit trail.
- Production boot with a placeholder AUTH_JWT_SECRET → refused to start.